SignalForge

AI-Powered Trading Signal System

Functional Specification Document

Document ID:	SF-FSD-001
Version:	1.0
Date:	June 25, 2026
Status:	Final
Prepared By:	Hermes Agent (AI Assistant)

. Table of Contents

1. System Overview

SignalForge is a single-process asynchronous Python application that runs continuously on a Linux server (AWS EC2). It connects to Binance via WebSocket for live price data, processes market data through a 12-layer analysis pipeline, and delivers high-confidence trading signals to the user via a Telegram bot.

1.1 Architectural Summary

```
Binance WS --> Layer 0:  Data Ingestion
                    |
               Layer 1:  Feature Engineering
                    |
               Layer 2:  Pattern & Structure Detection (SMC/ICT)
                    |
               Layer 3:  Confluence Scoring (12 categories)
                    |
               Layer 4:  MTF Bias Engine
                    |
               Layer 5:  Cooldown & Dedup Gate
                    |
               Layer 6:  LLM Reasoning Engine (DeepSeek)
                    |
               Layer 7:  Filter Gate (10 filters)
                    |
               Layer 8:  Risk & Sizing Engine
                    |
               Layer 9:  Execution & Delivery (Telegram)
                    |
          Layer 10-12:  Logging, Tracking, Health Monitoring
```

2. Module Specifications — Layer 0 to Layer 12

Layer 0 — Data Ingestion (data/websocket_feed.py)

Maintains persistent WebSocket connections to Binance for 4 assets (BTC, ETH, BNB, SOL). Receives real-time tick and candlestick data. Auto-reconnects on disconnect within 30s. Feeds normalized OHLCV data into the shared in-memory data store.

Layer 1 — Feature Engineering (signals/feature_engine.py)

Computes technical indicators: EMA 20/50/200, RSI 14, MACD (12,26,9), ATR 14, Bollinger Bands (20,2), Volume SMA 20, VWAP. Uses pandas-ta for vectorized computation.

Layer 2 — Pattern & Structure Detection (signals/smc_detector.py)

Implements SMC/ICT methodology: Swing High/Low detection, Break of Structure (BOS), Change of Structure (ChOS), Fair Value Gap (FVG) with mitigation tracking, Order Block detection (bullish/bearish), and Liquidity Grab identification.

Layer 3 — Confluence Scoring (signals/confluence_engine.py)

Scores 12 signal categories: trend alignment, SMC structure, FVG presence, order block proximity, liquidity sweep, volume confirmation, RSI alignment, MACD crossover, Bollinger Band squeeze, MTF agreement, session timing, and market structure. Raw score threshold: >= 8 (SOL: >= 9).

Layer 4 — MTF Bias Engine (signals/mtf_engine.py)

Evaluates 1H, 4H, and 1D timeframes for directional consensus. Bullish bias requires >= 2/3 timeframes aligned. Bearish bias same. Conflicting bias suppresses signal generation.

Layer 5 — Cooldown & Dedup Gate (signals/pipeline.py)

Enforces per-asset cooldown (default 4H) to prevent signal clustering. Deduplication checks last N signals for identical direction + asset within window.

Layer 6 — LLM Reasoning Engine (signals/llm_engine.py)

Submits structured market context JSON to DeepSeek-chat via OpenAI-compatible API. Receives JSON response with: action (LONG/SHORT/PASS), confidence (0-100), entry, stop_loss, take_profit, risk_reward, reasoning, key_risks. Falls back to json_repair on parse failure. Timeout: 30s.

Layer 7 — Filter Gate (signals/filter_gate.py)

10 sequential filters: (1) confidence >= 70%, (2) R:R >= 1.5, (3) not PASS action, (4) valid entry price, (5) stop_loss below entry for LONG, (6) take_profit above entry, (7) entry within 0.5% of current price, (8) ATR-based volatility check, (9) session whitelist (London/NY), (10) news blackout window.

Layer 8 — Risk & Sizing Engine (signals/risk_engine.py)

Calculates position size based on account equity, risk percentage (1% default), and distance to stop-loss. Output: quantity in base asset, notional value in USDT.

Layer 9 — Execution & Delivery (delivery/telegram_bot.py)

Formats signal as HTML Telegram message with emoji indicators. Sends via Bot API. Stores signal to SQLite. Updates paper trading engine.

Layer 10 — Logging & Tracking (monitoring/)

Structured JSON logging to rotating files. All signal events, filter decisions, LLM calls, and errors persisted. Daily log rotation with 7-day retention.

Layer 11 — Health Monitoring (monitoring/watchdog.py)

HTTP health endpoint on port 8080. Monitors WebSocket staleness (> 5min = alert), LLM API reachability, and disk space. Sends Telegram alert on any critical event.

Layer 12 — Paper Trading Engine (trading/paper_engine.py)

Simulates trade execution with $10,000 virtual capital. Tracks open positions, monitors TP/SL hits on every price tick, calculates unrealised and realised P&L.

3. Data Flow Architecture

End-to-end data movement from exchange to signal delivery:

- [1] Binance WebSocket: Streams real-time OHLCV candles + orderbook for BTC, ETH, BNB, SOL
- [2] In-Memory Buffer: Sliding window of last 500 candles per asset per timeframe
- [3] Feature Store: pandas DataFrame enriched with indicators, updated each candle
- [4] SMC Detector: Pattern flags appended to DataFrame: has_fvg, has_ob, bos_type, etc.
- [5] Confluence Engine: Scalar score (0–12) computed per asset per evaluation cycle
- [6] LLM Engine: Context dict serialised to JSON, submitted to DeepSeek API
- [7] Filter Gate: Binary pass/reject per filter; first failure short-circuits chain
- [8] SQLite: Signal record inserted with full context and LLM response
- [9] Telegram: HTML-formatted message sent to configured CHAT_ID
- [10] Paper Engine: Position opened with calculated size; monitored until TP/SL

4. API Specifications

4.1 Health Endpoint (port 8080)

Endpoint	Method	Response
/health	GET	200 OK + JSON: {status, uptime, ws_status, last_signal}
/health/ws	GET	200 OK + JSON: {btc, eth, bnb, sol} websocket staleness
/health/llm	GET	200 OK + JSON: {reachable, last_call_ms}

4.2 Telegram Bot Commands

Command	Response
/status	Current service status, uptime, last signal time
/signals	Last 5 signals with direction, asset, confidence, R:R
/trades	Open paper positions with unrealised P&L
/report	Weekly performance summary

/ping

Liveness check — responds with Pong + timestamp

5. Database Schema

9 SQLite tables in /home/ssm-user/signalforge/db/signalforge.db:

Table: signals

Columns: id, timestamp, asset, direction, confidence, rr, entry, sl, tp, reasoning, passed_filter

Table: paper_trades

Columns: id, signal_id, asset, direction, entry_price, quantity, sl, tp, status, pnl, opened_at, closed_at

Table: filter_log

Columns: id, signal_id, filter_name, passed, rejection_reason, timestamp

Table: llm_calls

Columns: id, signal_id, model, prompt_tokens, completion_tokens, latency_ms, timestamp

Table: ws_events

Columns: id, asset, event_type, timestamp, detail

Table: alerts

Columns: id, alert_type, message, sent, timestamp

Table: daily_summary

Columns: id, date, signals_evaluated, signals_passed, trades_opened, pnl

Table: config_audit

Columns: id, changed_by, parameter, old_value, new_value, timestamp

Table: health_checks

Columns: id, check_type, status, detail, timestamp

6. Error Handling & Edge Cases

Scenario	Behaviour
LLM API timeout (>30s)	Return PASS signal; log warning; increment timeout counter
LLM returns malformed JSON	Attempt json_repair; on failure return PASS; log raw response at DEBUG
WebSocket disconnect	ccxt.pro auto-reconnect; watchdog alerts if stale > 5min

Exchange API rate limit (429)

	Exponential backoff via ccxt; no crash

OOM kill by kernel

	systemd OOMPolicy=restart; service auto-restarts within 30s

SQLite lock contention

Retry with 0.1s backoff up to 3 attempts

Telegram API failure

	Log error; signal still saved to DB; retry next cycle

Invalid LLM price (0 or None)

Filter gate rejects at filter #4 (valid entry price check)

Confluence score below threshold

Signal suppressed before LLM call; logged as SKIP

7. Performance Requirements

Metric	Target	Observed
End-to-end signal latency	< 15s	~6–10s (LLM dominates)
LLM API response time	< 10s avg	~4s avg (DeepSeek-chat)
WebSocket tick processing	< 10ms	< 5ms
SQLite write latency	< 50ms	< 10ms
Memory footprint	< 1200MB	670–882MB
CPU usage (idle)	< 5%	~1–2%
CPU usage (peak, LLM call)	< 30%	~5–10%
Log file growth rate	< 500KB/h	~925KB/10min (verbose mode)

8. Configuration Reference

8.1 Environment Variables (config/.env)

Variable	Example Value	Purpose
TELEGRAM_BOT_TOKEN	[REDACTED]	Telegram Bot API token
TELEGRAM_CHAT_ID	[REDACTED]	Target chat for signal delivery
OPENAI_API_KEY	sk-...	LLM provider API key
OPENAI_BASE_URL	https://api.deepseek.com	LLM API endpoint
OPENAI_MODEL	deepseek-chat	Primary LLM model
OPENAI_FALLBACK	deepseek-chat	Fallback LLM model
PAPER_TRADING	true	Enable paper trading mode

8.2 Key Asset Config Parameters (config/assets.py)

Parameter	Default	Description
min_confluence_score	8 (SOL: 9)	Minimum score to trigger LLM evaluation
min_rr	1.5	Minimum risk:reward to pass filter gate
min_confidence	70	Minimum LLM confidence to pass filter gate
cooldown_hours	4	Hours between signals per asset
timeframes	[1h, 4h, 1d]	Timeframes analysed per asset

9. Approval

Role	Name	Date	Status
Product Owner	Fazrial	___/___/2026	Pending
Developer	Hermes Agent	25/06/2026	Ready
Reviewer		___/___/2026	Pending